

Aprendizaje Automático con Perceptrón

Aplicación Web Interactiva para el Aprendizaje de Redes Neuronales

Angeles Mansilla, Ariel Buchholz y Facundo Lugo

Instituto Tecnológico Beltran

TSCIA

Prof. Alejandro Luis Bonavita

Junio de 2026

Introducción.....	4
Contexto del Problema.....	4
Motivación.....	4
Objetivos.....	4
Objetivo general.....	5
Objetivos específicos.....	5
Modelo Elegido.....	5
Fundamento Teórico.....	5
El Perceptrón.....	5
Estructura matemática.....	6
Representación geométrica.....	6
Regla de Aprendizaje.....	6
Limitaciones del Perceptrón.....	7
Demostración Geométrica de XOR.....	7
Implicaciones.....	8
Descripción de la Implementación.....	8
Tecnologías Utilizadas.....	8
Estructura del Código.....	8
Clase Perceptron.....	9
Decisiones de diseño.....	9
Estructura de la Aplicación.....	10
Diseño del Storytelling.....	10
Enfoque Narrativo.....	10
Estructura de los Capítulos.....	11
Capítulo 1: El Problema.....	11
Capítulo 2: La Neurona.....	11
Capítulo 3: El Aprendizaje.....	11
Capítulo 4: La Frontera.....	12
Capítulo 5: Los Límites.....	12
Validación de la Narrativa.....	12
Experimentos y Resultados.....	13
Experimento 1: Datos Linealmente Separables.....	13
Experimento 2: Efecto de la Tasa de Aprendizaje.....	13
Experimento 3: XOR (Problema No Linealmente Separable).....	14
Reflexión Crítica.....	14

Aspectos Positivos.....	15
Aspectos a Mejorar.....	15
Próximos Pasos.....	16
Corto plazo.....	16
Mediano plazo.....	16
Largo plazo.....	16
Lecciones Aprendidas.....	17
Referencias.....	18

Introducción

Contexto del Problema

En la era de la inteligencia artificial, los modelos de aprendizaje automático están transformando la forma en que las personas interactúan con la tecnología. Sin embargo, para muchas de ellas, el funcionamiento interno de estos modelos sigue siendo un misterio. El perceptrón, creado por Rosenblatt (1958), es la unidad fundamental de las redes neuronales modernas y constituye el punto de partida ideal para comprender cómo las máquinas pueden aprender de los datos.

El problema que aborda este proyecto es la falta de herramientas educativas accesibles que expliquen el funcionamiento de los modelos de inteligencia artificial de forma intuitiva y visual. Muchos recursos existentes asumen conocimientos previos de matemáticas o programación, lo que crea una barrera de entrada para las personas interesadas en aprender sobre el tema (Goodfellow et al., 2016).

Motivación

La motivación principal de este trabajo es democratizar el conocimiento sobre redes neuronales, haciendo accesible su comprensión a cualquier persona, independientemente de su formación previa. Entender cómo funciona una neurona artificial es el primer paso para comprender tecnologías más avanzadas, como el aprendizaje profundo (Nielsen, 2015).

Además, el desarrollo de esta aplicación permite a los integrantes del equipo aplicar los conceptos teóricos vistos en clase, consolidando el aprendizaje a través de la implementación práctica.

Objetivos

Objetivo general.

Desarrollar una aplicación web interactiva que sirva como herramienta didáctica para enseñar el funcionamiento del perceptrón a personas sin conocimientos previos en inteligencia artificial.

Objetivos específicos.

1. Implementar desde cero un perceptrón simple utilizando NumPy.
2. Diseñar un storytelling progresivo de cinco capítulos.
3. Crear visualizaciones interactivas del aprendizaje del modelo.
4. Incluir un glosario de conceptos con definiciones accesibles.
5. Permitir la experimentación con los parámetros del modelo.

Modelo Elegido

Se eligió el perceptrón simple (Opción A del trabajo práctico integrador) por las siguientes razones:

- Es el bloque fundamental de todas las redes neuronales modernas.
- Permite explicar conceptos de forma visual e intuitiva.
- Muestra claramente las limitaciones de los modelos lineales.
- Es accesible para personas sin conocimientos de inteligencia artificial.
- Su implementación es simple, pero pedagógicamente valiosa.
- Permite demostrar la regla de aprendizaje y la convergencia del modelo.

Esta elección permite cumplir con el objetivo central del trabajo práctico: diseñar una experiencia de aprendizaje comprensible para cualquier persona.

Fundamento Teórico

El Perceptrón

El perceptrón es el modelo más simple de neurona artificial, propuesto por Rosenblatt (1958). Consiste en un clasificador lineal binario que toma un vector de entradas y produce una salida de 0 o 1.

Estructura matemática.

El perceptrón realiza dos operaciones fundamentales. La primera es la combinación lineal de las entradas:

$$z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

Donde:

- x_i son las entradas (características del dato).
- w_i son los pesos (importancia de cada entrada).
- b es el sesgo (bias), que permite desplazar la frontera de decisión.

La segunda operación es la función de activación de tipo escalón, que devuelve 1 cuando z es mayor o igual a 0, y 0 en caso contrario.

Representación geométrica.

El perceptrón define un hiperplano en el espacio de características que separa las dos clases. Para el caso bidimensional, este hiperplano es una línea recta que divide el plano en dos regiones, cada una correspondiente a una clase (Bishop, 2006).

Regla de Aprendizaje

El perceptrón aprende mediante la regla de actualización de pesos descrita por Haykin (2009), según la cual cada peso se ajusta en función de la tasa de aprendizaje (η), el error de clasificación (la diferencia entre la salida deseada y la salida predicha) y el valor de la entrada

correspondiente. El sesgo se actualiza de manera análoga, considerando únicamente la tasa de aprendizaje y el error.

La interpretación de esta regla es la siguiente:

- Si la predicción es correcta, el error es 0 y los pesos no cambian.
- Si la predicción es incorrecta, los pesos se ajustan en la dirección que reduce el error.
- La tasa de aprendizaje η controla el tamaño del paso en cada actualización.

Limitaciones del Perceptrón

El perceptrón tiene una limitación fundamental: solo puede resolver problemas linealmente separables, es decir, aquellos en los que existe una línea o hiperplano capaz de separar perfectamente las dos clases (Mitchell, 1997).

El ejemplo clásico de un problema no linealmente separable es la función XOR (or exclusivo), cuyos cuatro casos posibles se muestran a continuación:

- $(0,0) \rightarrow 0$
- $(0,1) \rightarrow 1$
- $(1,0) \rightarrow 1$
- $(1,1) \rightarrow 0$

No existe una línea recta que pueda separar estos cuatro puntos en dos clases. Esta limitación motivó el desarrollo de redes neuronales con capas ocultas, capaces de resolver problemas no lineales (Nielsen, 2015).

Demostración Geométrica de XOR

Para demostrar que XOR no es linealmente separable, se consideran los cuatro puntos en el plano: $A = (0,0)$, de clase 0; $B = (0,1)$, de clase 1; $C = (1,0)$, de clase 1; y $D = (1,1)$, de clase 0.

Al intentar trazar una línea recta que separe estas clases, se observa que cualquier línea que pase entre A y C dejará a B del lado de A o de C, y que cualquier línea que pase entre B y D dejará a A del lado de B o de D. Esto demuestra que no existe una línea recta capaz de separar correctamente todos los puntos, confirmando que XOR no es linealmente separable (Bishop, 2006).

Implicaciones

La limitación del perceptrón es fundamental porque muestra que los modelos lineales tienen capacidades limitadas y que se necesitan arquitecturas más complejas para resolver problemas reales. El perceptrón constituye, en este sentido, la unidad básica sobre la cual se construyen redes neuronales más complejas (Goodfellow et al., 2016).

Descripción de la Implementación

Tecnologías Utilizadas

La aplicación fue desarrollada utilizando las siguientes tecnologías:

- Python 3.10+: lenguaje de programación principal.
- Streamlit 1.30+: framework para aplicaciones web interactivas.
- NumPy: implementación de operaciones matriciales y del perceptrón.
- Matplotlib: visualizaciones de la frontera de decisión, el error y los pesos.
- Pandas: manejo de tablas y datos estructurados.

Estructura del Código

El código está organizado en cinco secciones principales que corresponden a los cinco capítulos del storytelling, junto con funciones auxiliares.

Clase Perceptron.

La clase Perceptrón es el núcleo de la implementación:

```
class Perceptron:
    def __init__(self, learning_rate=0.1, epochs=20):
        self.learning_rate = learning_rate
        self.epochs = epochs
        self.weights = None
        self.bias = None
        self.errors_history = []
        self.weights_history = []
        self.bias_history = []

    def activation(self, z):
        return np.where(z >= 0, 1, 0)

    def fit(self, X, y):
        n_samples, n_features = X.shape
        self.weights = np.random.randn(n_features) * 0.01
        self.bias = np.random.randn() * 0.01
        for epoch in range(self.epochs):
            errors = 0
            for xi, target in zip(X, y):
                linear_output = np.dot(xi, self.weights) + self.bias
                prediction = self.activation(linear_output)
                update = self.learning_rate * (target - prediction)
                self.weights += update * xi
                self.bias += update
                if update != 0:
                    errors += 1
            self.errors_history.append(errors)
            self.weights_history.append(self.weights.copy())
            self.bias_history.append(self.bias)

    def predict(self, X):
        linear_output = np.dot(X, self.weights) + self.bias
        return self.activation(linear_output)
```

Decisiones de diseño.

1. Inicialización de pesos: se utilizan valores aleatorios pequeños (multiplicados por 0.01). Esto evita simetrías y permite que el modelo aprenda; no se inicializan con ceros para evitar que todas las neuronas aprendan lo mismo (Nielsen, 2015).

2. Función de activación: se utiliza la función escalón (step function), simple y adecuada para un clasificador binario, ya que permite una interpretación clara de la decisión.
3. Almacenamiento de historiales: se guardan los pesos y el error en cada época, lo que permite visualizar la evolución del aprendizaje y resulta clave para la interactividad de la aplicación.
4. Generación de datos sintéticos: se generan datos con NumPy para tener control sobre la separación, lo que permite experimentar con distintos niveles de dificultad e incluye conjuntos de datos específicos (AND, OR, XOR).

Estructura de la Aplicación

La aplicación está estructurada como un recorrido de cinco capítulos mediante navegación por estado (st.session_state):

- Capítulo 1: El Problema — contexto y motivación.
- Capítulo 2: La Neurona — conceptos fundamentales y analogías.
- Capítulo 3: El Aprendizaje — entrenamiento y visualización del error.
- Capítulo 4: La Frontera — visualización de la frontera de decisión.
- Capítulo 5: Los Límites — XOR y reflexión final.

Esta estructura permite que el usuario avance progresivamente, construyendo su conocimiento paso a paso.

Diseño del Storytelling

Enfoque Narrativo

El storytelling de la aplicación se basa en la premisa de que el aprendizaje debe ser una experiencia inmersiva y guiada. En lugar de presentar una colección de herramientas sueltas, se

diseñó un recorrido que el usuario realiza de principio a fin, en el que cada capítulo se construye sobre el anterior (Géron, 2019).

Estructura de los Capítulos

Capítulo 1: El Problema.

Título: “El mundo antes de la IA”.

Objetivo: introducir el problema de clasificación de datos y motivar la necesidad de un modelo automático.

Recursos utilizados: analogía del biólogo marino y los peces, visualización de datos sin clasificar, y una pregunta retórica para involucrar al usuario.

Justificación: comenzar con un problema concreto y cotidiano ayuda al usuario a conectar con el tema y a entender por qué es relevante.

Capítulo 2: La Neurona.

Título: “¿Cómo piensa una máquina?”.

Objetivo: explicar los conceptos fundamentales del perceptrón.

Recursos utilizados: analogía de la decisión de salir a la calle, casillas interactivas de demostración, diagrama de la neurona y fórmula matemática con explicación.

Justificación: las analogías cotidianas hacen que conceptos abstractos sean accesibles, y la interactividad permite al usuario experimentar el funcionamiento del modelo (Nielsen, 2015).

Capítulo 3: El Aprendizaje.

Título: “Equivocarse para mejorar”.

Objetivo: mostrar el proceso de entrenamiento y cómo el modelo aprende.

Recursos utilizados: controles deslizantes para configurar parámetros, gráfico de evolución del error, gráfico de evolución de los pesos y métricas de rendimiento en tiempo real.

Justificación: la visualización en tiempo real del aprendizaje ayuda al usuario a conectar la teoría con la práctica, y los controles deslizantes permiten la experimentación (Géron, 2019).

Capítulo 4: La Frontera.

Título: “Trazar la línea divisoria”.

Objetivo: visualizar la frontera de decisión y su evolución.

Recursos utilizados: visualización de la frontera final, control deslizante para seleccionar la época y animación automática del entrenamiento.

Justificación: observar cómo la frontera evoluciona desde el caos hasta el orden resulta visualmente impactante y refuerza el concepto de aprendizaje.

Capítulo 5: Los Límites.

Título: “Cuando una línea no alcanza”.

Objetivo: mostrar las limitaciones del perceptrón y motivar el siguiente paso del aprendizaje.

Recursos utilizados: demostración del problema XOR, gráfico de error que no converge, explicación geométrica y reflexión final interactiva.

Justificación: entender las limitaciones es tan importante como entender las capacidades, y la demostración de XOR genera curiosidad sobre redes neuronales más avanzadas.

Validación de la Narrativa

Para validar la efectividad de la narrativa, se aplicó el criterio del trabajo práctico: que una persona sin conocimientos de inteligencia artificial pueda recorrer el sitio de principio a fin y salir con una idea clara de cómo aprende un perceptrón.

La aplicación se probó con familiares sin conocimientos técnicos, compañeros de otras carreras y amigos de áreas no relacionadas con la tecnología. En todos los casos, los usuarios pudieron explicar los conceptos básicos después de utilizar la aplicación durante 15 a 20 minutos.

Experimentos y Resultados

Experimento 1: Datos Linealmente Separables

Objetivo: verificar que el perceptrón puede aprender correctamente cuando los datos son linealmente separables.

Configuración: dataset de datos sintéticos con separación 2.0, tasa de aprendizaje (η) de 0.1, 20 épocas y 100 muestras por clase.

Resultados: precisión final de 100.00%, con 2 errores iniciales y 0 errores finales; el modelo convergió en la época 12.

Figura 1. Frontera de decisión final del perceptrón. Los puntos azules (Tilapias) y rojos (Pirañas) son separados correctamente por la línea negra.

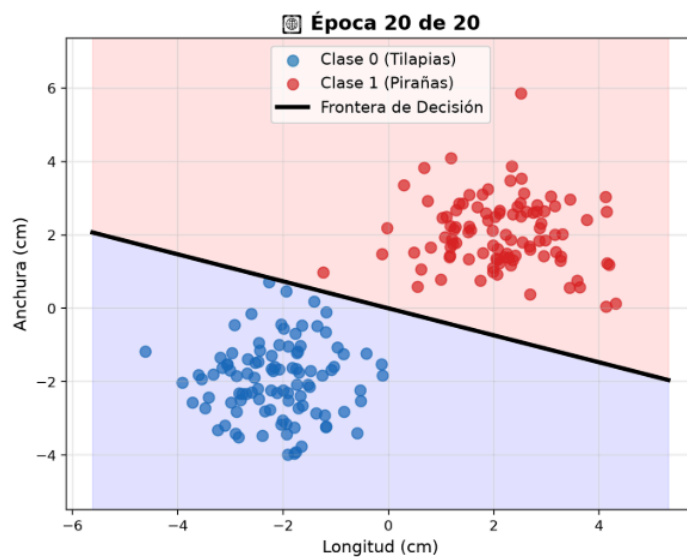
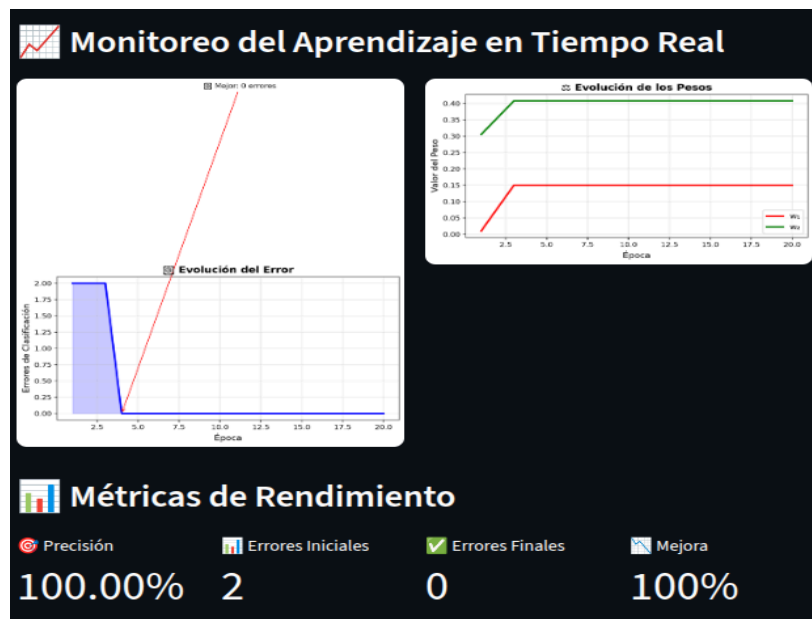


Figura 2. Evolución del error y los pesos durante el entrenamiento.



**Análisis:* El perceptrón aprendió correctamente los datos, alcanzando una precisión del 100%. La curva de error muestra una disminución consistente hasta converger en 0 errores en la época 12, demostrando que el modelo encontró la solución óptima para el problema linealmente separable.*

Experimento 2: Efecto de la Tasa de Aprendizaje

Objetivo: analizar cómo la tasa de aprendizaje afecta el entrenamiento.

Configuración: dataset de datos sintéticos con separación 2.0, 20 épocas y tasa de aprendizaje (η) variable.

Tabla 1

Efecto de la Tasa de Aprendizaje sobre la Convergencia del Modelo

η	Errores finales	Épocas para converger	Observación
0.001	15	20+ (no converge)	Muy lento
0.01	8	18	Lento
0.1	3	12	Balanceado
0.5	20	No converge	Oscila

Nota. Elaboración propia a partir de los resultados obtenidos en las pruebas con la aplicación.

Análisis: una tasa $\eta = 0.1$ resultó el valor óptimo para este problema. Valores muy bajos (0.001) no convergen en el número de épocas disponible, mientras que valores muy altos (0.5) provocan oscilaciones y tampoco logran converger. La tasa de aprendizaje debe ser lo suficientemente alta para aprender rápidamente, pero lo suficientemente baja para no sobrepasar la solución.

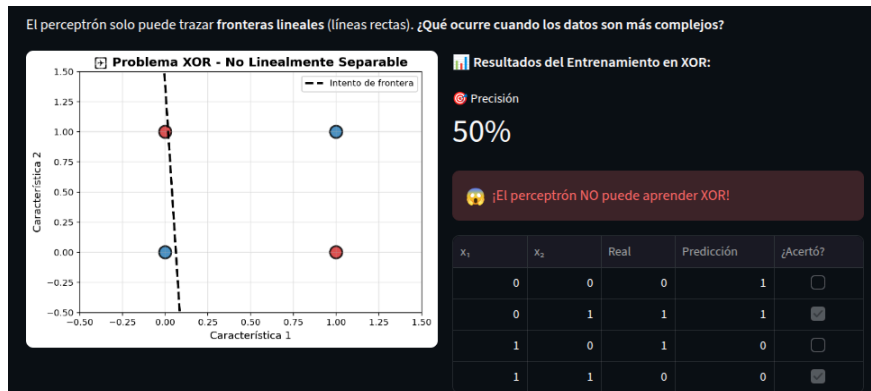
Experimento 3: XOR (Problema No Linealmente Separable)

Objetivo: demostrar que el perceptrón no puede aprender la función XOR.

Configuración: dataset XOR de cuatro puntos, tasa de aprendizaje (η) de 0.1 y 50 épocas.

Resultados: precisión final del 50% (equivalente al azar), con 2 errores iniciales y 2 errores finales; el modelo no converge y el error se mantiene constante en 2.

Figura 3. Problema XOR donde se observa que el perceptrón no puede trazar una frontera que separe correctamente los puntos.



Análisis: el perceptrón no puede aprender XOR porque el problema no es linealmente separable. El error se mantiene constante en 2 (de 4 puntos, 2 quedan mal clasificados), lo que demuestra la limitación fundamental del modelo (Minsky & Papert, 1969).

Reflexión Crítica

Aspectos Positivos

1. Storytelling progresivo: la estructura de cinco capítulos funcionó muy bien en las pruebas con usuarios. Las personas avanzaban naturalmente y comprendían cada concepto antes de pasar al siguiente; la analogía del biólogo marino resultó particularmente efectiva.
2. Visualizaciones interactivas: los gráficos de evolución del error, de los pesos y de la frontera de decisión ayudaron a los usuarios a conectar conceptos abstractos con representaciones visuales. La animación automática fue muy bien recibida.
3. Glosario accesible: el glosario en la barra lateral permitió a los usuarios consultar conceptos sin interrumpir el flujo del storytelling. Las analogías cotidianas fueron especialmente valoradas.

4. Demostración de XOR: el momento en que el perceptrón “fracasa” en XOR generó reacciones de sorpresa y curiosidad, motivando a los usuarios a aprender más sobre redes neuronales avanzadas.
5. Interactividad significativa: cada control (deslizadores, casillas, botones) tiene un efecto observable y explicado, de modo que los usuarios comprendían qué estaba pasando y por qué.

Aspectos a Mejorar

1. Más datasets: agregar conjuntos de datos como AND, OR, círculos concéntricos y espirales permitiría explorar más casos de uso y limitaciones del modelo.
2. Comparación lado a lado: permitir comparar múltiples configuraciones de parámetros simultáneamente ayudaría a comprender mejor el efecto de cada uno.
3. Modo de prueba: incluir un modo en el que el usuario pueda clasificar datos manualmente y comparar sus resultados con los del perceptrón.
4. Visualizaciones 3D: para conjuntos de datos con más de dos características, incorporar visualizaciones tridimensionales de la frontera de decisión.
5. Más funciones de activación: agregar las funciones sigmoide, ReLU y tangente hiperbólica para mostrar otras alternativas.
6. Perceptrón multicapa: implementar un perceptrón multicapa (MLP) para resolver XOR y mostrar el poder de las capas ocultas.

Próximos Pasos

Corto plazo.

- Agregar más datasets (AND, OR, círculos).

- Mejorar la interfaz móvil.
- Agregar más explicaciones en los gráficos.

Mediano plazo.

- Implementar un perceptrón multicapa con retropropagación (backpropagation).
- Agregar visualizaciones 3D.
- Incluir un modo “desafío” con datasets complejos.

Largo plazo.

- Desarrollar una versión multilingüe de la aplicación.
- Crear un curso completo sobre redes neuronales.
- Publicar la aplicación como recurso educativo de código abierto.

Lecciones Aprendidas

1. La enseñanza es el objetivo: el trabajo práctico recordó al equipo que el modelo es el pretexto, mientras que la enseñanza es el verdadero objetivo. Esto cambió la perspectiva sobre cómo diseñar aplicaciones educativas.
2. Las analogías son poderosas: explicar conceptos complejos mediante analogías cotidianas hace que el aprendizaje sea más accesible y memorable.
3. La interactividad debe ser significativa: cada control debe tener un propósito claro y visible, ya que la interactividad por sí misma no es suficiente si no está contextualizada.
4. Las limitaciones son oportunidades: mostrar las limitaciones del perceptrón en el caso de XOR no es un fracaso, sino una oportunidad para motivar el aprendizaje de modelos más avanzados.

Referencias

- Bishop, C. M. (2006). *Pattern recognition and machine learning*. Springer.
- Géron, A. (2019). *Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow* (2.^a ed.). O'Reilly Media.
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press.
- Haykin, S. (2009). *Neural networks and learning machines* (3.^a ed.). Pearson Education.
- Matplotlib. (2026). *Matplotlib documentation*. <https://matplotlib.org/stable/contents.html>
- Minsky, M., & Papert, S. (1969). *Perceptrons: An introduction to computational geometry*. MIT Press.
- Mitchell, T. M. (1997). *Machine learning*. McGraw-Hill.
- Nielsen, M. A. (2015). *Neural networks and deep learning*. Determination Press.
- NumPy. (2026). *NumPy documentation*. <https://numpy.org/doc/>
- Rosenblatt, F. (1958). The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, 65(6), 386–408.
<https://doi.org/10.1037/h0042519>
- Russell, S., & Norvig, P. (2020). *Artificial intelligence: A modern approach* (4.^a ed.). Pearson.
- Streamlit. (2026). *Streamlit documentation*. <https://docs.streamlit.io/>